

CPS188: Lab 06 - Arrays
Name: Iqraa Wahid
Student Number: 501287410

Problem #1:

Algorithm:

1. Read the first three numbers from the file, let n be the first number, and next two numbers be min and max respectively
2. Read the next n element in the textfile and store them in the array called array
3. Using the formula: $\min + (\text{array}[i] - \text{array_min}) * (\text{max} - \text{min}) / (\text{array_max} - \text{array_min})$, calculate the norm for each element in the array, array
4. Complete the above steps until the entire file has been read
5. Output the elements in the array, array, and their corresponding norm values in a column

Code:

```
#include <stdio.h>

int main (void){
    int integer;
    double min, max;

    FILE *myfile; //declaring file
    myfile=fopen("data.txt", "r"); //opening file in reading mode

    while (!feof(myfile)){ //while the end of file hasn't reached,
keep reading
        fscanf(myfile, "%d %lf %lf", &integer, &min, &max);
        if(integer<3 || integer>20){
            printf("The number n is not between 3 and 20, change
the numbers.");
            fclose(myfile); //close file if n is less than 3 or
larger than 20
            return(1);
        }

        double array [integer]; //declare array to store n real
numbers
        double norm [integer]; //declare array to store the n real
number's norms
        for(int i=0;i<integer;i++){
            fscanf(myfile, "%lf", &array[i]);
        }
    }
}
```

```

        //let the minimum and maximum element of the array be the
array at element 0
        double array_min=array[0];
        double array_max=array[0];

        //find the maximum and minimum value element of the array
        for (int i=1;i<integer;i++){
            if(array[i]<array_min){
                array_min=array[i];
            }
            if(array[i]>array_max){
                array_max=array[i];
            }
        }

        //find the norm of each element in the array
        for (int i=0;i<integer;i++){
            norm[i] = min + (((array[i] - array_min) * (max -
min)) / (array_max-array_min));
        }

        //output array elements and corresponding norms:
        printf("\n%-10s %-10s\n", "Original", "Normalized");
        printf("-----\n");
        for (int i=0;i<integer;i++) {
            printf("%-10.2f %-10.2f\n", array[i], norm[i]);
        }
    }
    fclose(myfile); //closing file
    return(0);
}

```

Output:

Original	Normalized
67.90	7.75
45.20	4.47
33.30	2.75
66.10	7.49
83.50	10.00
14.30	0.00

50.50 5.23

Original	Normalized
----------	------------

6.90	0.65
4.20	0.34
3.30	0.23
6.10	0.56
8.50	0.84
1.30	0.00
5.50	0.49
9.90	1.00
8.00	0.78
3.60	0.27
2.80	0.17

Original	Normalized
----------	------------

-34.30	33.75
50.90	100.00
0.00	60.42
43.20	94.01
-77.70	0.00

Screenshot:

Original	Normalized
67.90	7.75
45.20	4.47
33.30	2.75
66.10	7.49
83.50	10.00
14.30	0.00
50.50	5.23

Original	Normalized
6.90	0.65
4.20	0.34
3.30	0.23
6.10	0.56
8.50	0.84
1.30	0.00
5.50	0.49
9.90	1.00
8.00	0.78
3.60	0.27
2.80	0.17

Original	Normalized
-34.30	33.75
50.90	100.00
0.00	60.42
43.20	94.01
-77.70	0.00

...Program finished with exit code 0
Press ENTER to exit console.

Problem #2:

Code:

```
#include <stdio.h>
#include <stdlib.h>

// for integers (ascending)
int integer_ascending(const void*a, const void*b){
    return (*(int*)a-*(int*)b);
}

// for double (ascending)
int double_ascending(const void*a, const void*b){
    if(*(double*)a>*(double*)b) return 1;
    if(*(double*)a<*(double*)b) return -1;
    return (0);
}

// for integers (descending)
int integer_descending(const void*a, const void*b){
    return (*(int*)b-*(int*)a);
}

// for double (descending)
int double_descending(const void*a, const void*b){
    if(*(double*)a<*(double*)b) return 1;
    if(*(double*)a>*(double*)b) return -1;
    return (0);
}

void print_int_array(int a [], int size){
    int i;
    for(i=0;i<size;i++){
        printf("%5d", a[i]);
    }
    printf("\n");
}

void print_double_array(double a [], double size){
    int i;
    for(i=0;i<size;i++){
        printf("%.2lf", a[i]);
    }
}
```

```

    }
    printf("\n");
}

int main(void){
    FILE *myfile; //open file
    int int_array[100]; //declare an array to store integer values
    int int_count = 0;
    double double_array[100]; //declare an array to store double
values
    int double_count=0;
    double ref;

    myfile=fopen("data.txt", "r"); //open file in reading mode
    while(fscanf(myfile, "%lf", &ref)==1){
        //storing integer values in the integer array
        if((int)ref==ref){
            int_array[int_count++]=(int)(ref);
        }
        //storing double values in the double array
        else{
            double_array[double_count++]=ref;
        }
    }
    fclose(myfile); //closing file

    //sorting integer values in ascending order
    qsort(int_array, int_count, sizeof(int), integer_ascending);
    //output
    printf("Array of integers in ascending order:\n");
    print_int_array(int_array, int_count);

    //sorting double values in ascending order
    qsort(double_array, double_count, sizeof(double),
double_ascending);
    //output
    printf("Array of doubles in ascending order:\n");
    print_double_array(double_array, double_count);

    //sorting integer values in descending order
    qsort(int_array, int_count, sizeof(int), integer_descending);

```

```

    //output
    printf("Array of integers in descending order:\n");
    print_int_array(int_array, int_count);

    //sorting double values in descending order
    qsort(double_array, double_count, sizeof(double),
double_descending);
    //output
    printf("Array of doubles in descending order:\n");
    print_double_array(double_array, double_count);
}

```

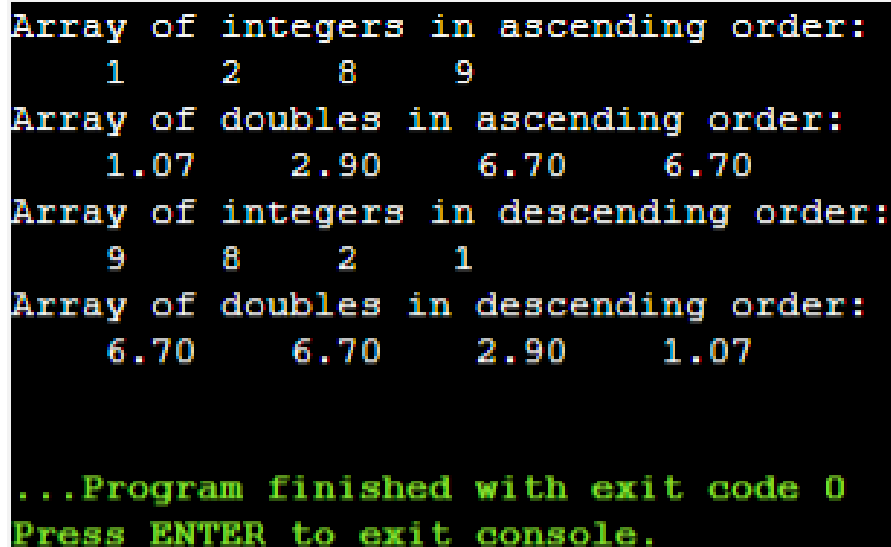
Output:

```

Array of integers in ascending order:
    1    2    8    9
Array of doubles in ascending order:
    1.07    2.90    6.70    6.70
Array of integers in descending order:
    9    8    2    1
Array of doubles in descending order:
    6.70    6.70    2.90    1.07

```

Screenshot:



```

Array of integers in ascending order:
    1    2    8    9
Array of doubles in ascending order:
    1.07    2.90    6.70    6.70
Array of integers in descending order:
    9    8    2    1
Array of doubles in descending order:
    6.70    6.70    2.90    1.07

...Program finished with exit code 0
Press ENTER to exit console.

```

Problem #3:

Code:

```

#include <stdio.h>

```

```

#define STACK_EMPTY '0'
#define STACK_SIZE 20

void push(char stack[],char item, int *top, int max_size){
    if (*top < max_size-1) {
        ++(*top);
        stack[*top] = item;
    }
}

char pop (char stack[], int *top){
    char item;
    if (*top >= 0) {
        item = stack[*top];
        --(*top);

    }
    else {
        item = STACK_EMPTY;
    }

    return (item);
}

//function to output stack
void printStack(char stack[], int top){
    if(top<0){
        printf("Stack is empty.\n");
        return;
    }

    printf("Stack: ");
    for(int i=0;i<=top;i++){
        printf("%c ", stack[i]);
    }
    printf("\n");
}

int main (void) {
    char s [STACK_SIZE];
    int s_top = -1; // stack is empty

```



```
//call push function five times
push(s, 'A', &s_top, STACK_SIZE);
push(s, 'B', &s_top, STACK_SIZE);
push(s, 'C', &s_top, STACK_SIZE);
push(s, 'D', &s_top, STACK_SIZE);
push(s, 'E', &s_top, STACK_SIZE);

//output updated stack
printStats(s, s_top);

//call pop function twice
pop(s, &s_top);
pop(s, &s_top);

//output updated stack
printStats(s, s_top);

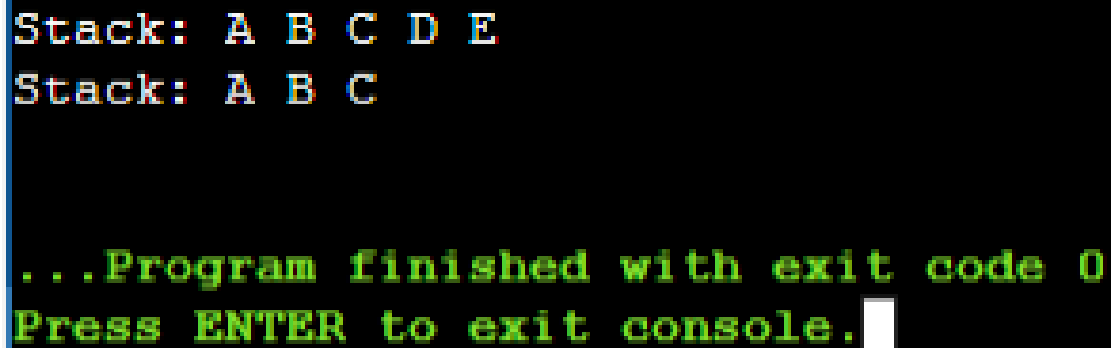
return (0);
}
```

Output:

Stack: A B C D E

Stack: A B C

Screenshot:



```
Stack: A B C D E
Stack: A B C

...Program finished with exit code 0
Press ENTER to exit console.
```